

1. 절차적 프로그래밍 (Procedural Programming)

✅ 특성

프로그램을 순서대로 실행되는 명령어 집합으로 바라봄.

데이터와 함수를 분리하고, 함수를 호출하는 흐름(절차) 중심.

구조화된 프로그래밍의 기반 (예: C, Fortran, Pascal).

👍 장점

코드 흐름이 직관적이고 이해하기 쉬움.

단순한 문제를 빠르게 해결하기 적합.

메모리나 실행 속도 최적화에 유리 (낮은 레벨 제어).

👎 단점

규모가 커질수록 코드가 복잡해지고 중복 코드가 많아짐.

데이터와 함수가 분리되어 있어 유지보수성 떨어짐.

모듈화·재사용성 한계.

📊 데이터분석 실무 활용

스크립트성 작업 (데이터 전처리 자동화, 파일 읽기/쓰기, 로그 분석 등)

단발성 분석 코드 (한 번 돌려보는 실험, 간단한 집계 작업)

예: pandas로 CSV를 불러와 집계 후 바로 저장하는 파이프라인

절차적 스타일

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")  
df = df.dropna()  
df["sales"] = df["price"] * df["quantity"]  
df.to_csv("cleaned.csv", index=False)
```

2. 함수형 프로그래밍 (Functional Programming, FP)

✅ 특성

순수 함수(Pure Function) 중심: 입력 → 출력 (부작용 없음).

불변성(Immutability) 강조.

함수 조합·고차 함수(map, filter, reduce) 적극 활용.

대표 언어: Haskell, Clojure, Elixir, (Python, R, Scala, JavaScript 일부 지원).

👍 장점

데이터 상태 변화가 없어서 병렬 처리·분산 처리에 안정적.

코드가 수학적이고 간결함 → 데이터 변환 로직 표현이 직관적.

테스트 용이 (동일 입력 → 동일 출력).

💡 단점

초보자에게 추상적(람다, 고차 함수, 모나드 등).

지나치게 함수 조합식으로만 작성하면 가독성 저하.

상태 기반 문제(예: 시뮬레이션, UI 관리)는 불편.

📊 데이터분석 실무 활용

데이터 변환 파이프라인 (ETL, 데이터 클렌징, 전처리)

빅데이터 처리: Spark, MapReduce 같은 분산 처리 프레임워크는 함수형 기반

함수형 라이브러리: Python map, filter, reduce, Pandas의 apply → 병렬화에도 활용

함수형 스타일

```
from functools import reduce

data = [1, 2, 3, 4, 5]

# map + filter + reduce 활용
squared_sum = reduce(lambda x, y: x + y,
                      map(lambda n: n**2,
                          filter(lambda n: n % 2 == 1, data)))
print(squared_sum) # 1^2 + 3^2 + 5^2 = 35
```

3. 객체지향 프로그래밍 (Object-Oriented Programming, OOP)

✅ 특성

객체(Object) = 데이터(속성) + 메서드(행동)의 결합.

캡슐화, 상속, 다형성을 통한 확장성/재사용성 강조.

대표 언어: Java, C++, Python, C#.

👍 장점

현실 세계 모델링에 적합 → 구조적이고 직관적.

대규모 프로젝트에 강함 (코드 재사용, 확장, 유지보수 용이).

모듈화가 잘 되어 협업에 유리.

🗨 단점

작은 문제에도 객체 설계가 과도하면 오히려 비효율적.

객체 간 의존성이 복잡해지면 디버깅이 어려움.

실행 성능은 절차적에 비해 다소 떨어질 수 있음.

🌈 데이터분석 실무 활용

분석 도구/프레임워크 개발 (클래스 기반 구조 설계)

ML 모델 객체화: scikit-learn의 모델(LinearRegression, RandomForestClassifier)은 OOP 기반

데이터 파이프라인 관리: Airflow, Luigi 등 워크플로우 관리 툴도 OOP 구조

OOP 스타일

```
import pandas as pd

class DataProcessor:
    def __init__(self, filename):
        self.df = pd.read_csv(filename)
    def clean(self):
        self.df = self.df.dropna()
    def add_sales(self):
        self.df["sales"] = self.df["price"] * self.df["quantity"]
    def save(self, out):
```

```
self.df.to_csv(out, index=False)

processor = DataProcessor("data.csv")
processor.clean()
processor.add_sales()
processor.save("cleaned.csv")
```

4. 데이터분석 실무 종합 활용

절차적: 작은 데이터 처리, 빠른 프로토타입, 자동화 스크립트에 적합

함수형: 대규모 데이터 변환, 분산 처리(Spark, Hadoop), 수학적 데이터 처리에 적합

객체지향: 분석 도구 개발, 머신러닝 모델 구조화, 데이터 파이프라인 관리에 적합

👉 실무에서는 혼합 사용이 일반적입니다. 예를 들어,

데이터 전처리는 함수형 스타일 (pandas.apply, map)

ML 모델링은 OOP 스타일 (scikit-learn 클래스)

분석 자동화 스크립트는 절차적 스타일

◆ 정리하면,

절차적 = 빠르고 단순, 스크립트성에 강함

함수형 = 데이터 변환과 분산 처리에 강함

객체지향 = 구조화와 재사용, 대규모 협업에 강함